

AFRILANG Stdlib

std/c/graphbellman

Français. Bibliothèque AFRILANG 'std/c/graphbellman' (niveau complexe, catégorie complex). Elle expose 6 fonction(s) : bellmanFord, hasNegCycle, negEdgeCount, relaxOnce, pathCost, reachableWeighted.

'import "std/c/graphbellman"' · 'use graphbellman'

- 'bellmanFord(adj list number, n number, start number) → list number'
- 'hasNegCycle(adj list number, n number) → bool' - 'negEdgeCount(adj list number, n number) → number' - 'relaxOnce(adj list number, n number, dist list number) → list number' - 'pathCost(adj list number, n number, path list number) → number' - 'reachableWeighted(adj list number, n number, start number) → list number'

English. AFRILANG library 'std/c/graphbellman' (tier complex, category complex). Exports 6 function(s): bellmanFord, hasNegCycle, negEdgeCount, relaxOnce, pathCost, reachableWeighted.

'import "std/c/graphbellman"' · 'use graphbellman'

- 'bellmanFord(adj list number, n number, start number) → list number'
- 'hasNegCycle(adj list number, n number) → bool' - 'negEdgeCount(adj list number, n number) → number' - 'relaxOnce(adj list number, n number, dist list number) → list number' - 'pathCost(adj list number, n number, path list number) → number' - 'reachableWeighted(adj list number, n number, start number) → list number'

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/graphbellman' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : `bellmanFord`, `hasNegCycle`, `negEdgeCount`, `relaxOnce`, `pathCost`, `reachableWeighted`.

'import "std/c/graphbellman"' · 'use graphbellman'

```
- `bellmanFord(adj list number, n number, start number) → list number`
- `hasNegCycle(adj list number, n number) → bool`
- `negEdgeCount(adj list number, n number) → number`
- `relaxOnce(adj list number, n number, dist list number) → list number`
- `pathCost(adj list number, n number, path list number) → number`
- `reachableWeighted(adj list number, n number, start number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphbellman"
use graphbellman
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- `bellmanFord(adj list number, n number, start number) → list`
- `hasNegCycle(adj list number, n number) → bool`
- `negEdgeCount(adj list number, n number) → number`
- `relaxOnce(adj list number, n number, dist list number) → list`
- `pathCost(adj list number, n number, path list number) → number`
- `reachableWeighted(adj list number, n number, start number) → number`

4. Exemples d'utilisation

```
import "std/c/graphbellman"
use graphbellman
```

```
create result = bellmanFord(/* args */)
say result
```

```
create result = hasNegCycle(/* args */)
say result
```

```
create result = negEdgeCount(/* args */)
say result
```

```
create result = relaxOnce(/* args */)
say result
```

```
create result = pathCost(/* args */)
say result
```

```
create result = reachableWeighted(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/graphbellman"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module graphbellman

```
function bellmanFord(adj list number, n number, start number) returns list number  
end
```

```
function hasNegCycle(adj list number, n number) returns bool  
end
```

```
function negEdgeCount(adj list number, n number) returns number  
end
```

```
function relaxOnce(adj list number, n number, dist list number) returns list number  
end
```

```
function pathCost(adj list number, n number, path list number) returns number  
end
```

```
function reachableWeighted(adj list number, n number, start number) returns number  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/graphbellman' (tier complex, category complex). Exports 6 function(s): bellmanFord, hasNegCycle, negEdgeCount, relaxOnce, pathCost, reachableWeighted.

```
'import "std/c/graphbellman"' · 'use graphbellman'
```

```
- `bellmanFord(adj list number, n number, start number) → list number`
- `hasNegCycle(adj list number, n number) → bool`
- `negEdgeCount(adj list number, n number) → number`
- `relaxOnce(adj list number, n number, dist list number) → list number`
- `pathCost(adj list number, n number, path list number) → number`
- `reachableWeighted(adj list number, n number, start number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphbellman"
use graphbellman
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- bellmanFord(adj list number, n number, start number) → list•
- hasNegCycle(adj list number, n number) → bool•
- negEdgeCount(adj list number, n number) → number•
- relaxOnce(adj list number, n number, dist list number) → list•
- pathCost(adj list number, n number, path list number) → number•
- reachableWeighted(adj list number, n number, start number) → number

4. Usage examples

```
import "std/c/graphbellman"
use graphbellman
```

```
create result = bellmanFord(/* args */)
say result
```

```
create result = hasNegCycle(/* args */)
say result
```

```
create result = negEdgeCount(/* args */)
say result
```

```
create result = relaxOnce(/* args */)
say result
```

```
create result = pathCost(/* args */)
say result
```

```
create result = reachableWeighted(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/graphbellman"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module graphbellman

```
function bellmanFord(adj list number, n number, start number) returns list number
end
```

```
function hasNegCycle(adj list number, n number) returns bool
end
```

```
function negEdgeCount(adj list number, n number) returns number
end
```

```
function relaxOnce(adj list number, n number, dist list number) returns list number
end
```

```
function pathCost(adj list number, n number, path list number) returns number
end
```

```
function reachableWeighted(adj list number, n number, start number) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/graphbellman"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/graphbellman/>

Source (extrait)

```
module graphbellman

function bellmanFord(adj list number, n number, start number) returns list number
end

function hasNegCycle(adj list number, n number) returns bool
end

function negEdgeCount(adj list number, n number) returns number
end

function relaxOnce(adj list number, n number, dist list number) returns list number
end

function pathCost(adj list number, n number, path list number) returns number
```

```
end
```

```
function reachableWeighted(adj list number, n number, start number) returns number
```

```
end
```

```
end
```